In [1]:
```
import numpy as np
import sisl
from sisl.viz.processors.math import normalize
from utils.loader import load_datastructure, path_finder, parse_lwc
from utils.plots import mark_electrode
from utils.structure import find_nearest_atoms

from ipywidgets import interact
```

In [2]:
```
SCALE = 0.6
OFFSET = 0.1
def plot_pdos_old(lwc):
    PARAMS = "{}L_{}W_{}C".format(*lwc)
    # extract data
    DATA = np.load(f"../notebooks/calculations/device_ldos_{PARAMS}.npz")
    ldos, lr_idx = DATA["ldos"], DATA["lr_idx"]
    # build device
    device = sisl.io.get_sile(f"../notebooks/structures/device_{PARAMS}.xyz").read_geometry()
    # scale pdos for plotting
    norm_pdos_avg_E = normalize(ldos.mean(axis=(0,1)))*0.6 + 0.1
    style = mark_electrode(lr_idx)
    style.append({"size": norm_pdos_avg_E})
    return device.plot(axes="xy", atoms_style=style)
```

In [3]:
```
def show_ldos_pdos_old(LWC):

    project_dir = path_finder("QTM")
    results_dir = project_dir / "results"
    notebook_dir = project_dir / "notebooks"
    atol = 0.01
    l, w, c = LWC
    _params_ = f"{l}L_{w}W_{c}C"
    # load data
    data = np.load(notebook_dir / f"calculations/device_ldos_{_params_}.npz")
    device = sisl.io.get_sile(notebook_dir / f"structures/device_{_params_}.xyz").read_geometry()
    site = find_nearest_atoms(device.xyz, device.center(), 1)
    ldos, lr_idx = data["ldos"], data["lr_idx"]
    ldos_center = ldos[0, :, site]
    energies = data["energies"]
    near_fermi = np.abs(energies) <= 0.5
    norm_pdos_avg_E = normalize(ldos.mean(axis=(0,1)),0, 3)
    bg_idxs = np.argwhere((normalize(ldos_center) < atol) & near_fermi)
    bgmin = min(energies[bg_idxs])
    bgmax = max(energies[bg_idxs])
    style = mark_electrode(lr_idx)
    style.append({"size": norm_pdos_avg_E})

    # 1. Generate the initial sisl plot (Spatial)
    fig = device.plot(axes="xy", atoms_style=style, backend="matplotlib")
    # 2. Resize the figure to be wider to accommodate two plots
    fig.set_size_inches(12, 6)

    # 3. Adjust the existing sisl axis to occupy only the left half
    # The first axis is usually fig.axes[0]
    ax_spatial = fig.gca()
    ax_spatial.set_position([0.05, 0.15, 0.4, 0.75])

    # 4. Add the new LDOS energy axis
    ax_energy = fig.add_axes([0.55, 0.15, 0.4, 0.75])
    ax_energy.plot(ldos_center, energies, color='blue')
    ax_energy.axhline(y=bgmin, color='red', linestyle='--', label='BG Min/Max')
    ax_energy.axhline(y=bgmax, color='red', linestyle='--')
    ax_energy.axvline(x=0, color='black', linestyle='--')
    ax_energy.set_title("LDOS vs Energy", fontsize=16)
    ax_energy.set_xlabel("LDOS (a.u.)", fontsize=14)
    ax_energy.set_ylabel("$E - E_f$ (eV)", fontsize=14)
    ax_energy.tick_params(axis='both', which='major', labelsize=10)

    # Apply sizes to Spatial Axis
    ax_spatial.tick_params(axis='both', which='major', labelsize=10)
    ax_spatial.set_xlabel(ax_spatial.get_xlabel(), fontsize=14)
    ax_spatial.set_ylabel(ax_spatial.get_ylabel(), fontsize=14)


    fig.suptitle(f"LDOS for LWC = {LWC}", fontsize=20)
```

In [4]:
```
def show_modulation(LWC, C):
    electrode, ribbon, device, data = load_datastructure(LWC)
    site = find_nearest_atoms(device.xyz, device.center(), 1)
    ldos = data[f"ldos_{C}" if C!="C0" else "ldos"]
    ldos_center = ldos[0, :, site]
    lr_idx = data["lr_idx"]
    energies = data["energies"]
    bgmin = data[f"bgmin_{C}"]
    bgmax = data[f"bgmax_{C}"]
    norm_pdos_avg_E = normalize(data["ldos"].mean(axis=(0,1)),0, 3)
    style = mark_electrode(lr_idx)
```

```
    style.append({"size": norm_pdos_avg_E})



    # 1. Generate the initial sisl plot (Spatial)
    fig = device.plot(axes="xy", backend="matplotlib", atoms_style=style)
    fig.suptitle(f"LWC = {LWC} with" + " {} ".format(C if C!="C0" else "no") + "modulation", fontsize=20)

    # 2. Resize the figure to be wider to accommodate two plots
    fig.set_size_inches(12, 6)

    # 3. Adjust the existing sisl axis to occupy only the left half
    # The first axis is usually fig.axes[0]
    ax_spatial = fig.gca()
    ax_spatial.set_position([0.05, 0.15, 0.4, 0.75])


    # 4. Add the new LDOS energy axis
    ax_energy = fig.add_axes([0.55, 0.15, 0.4, 0.75])
    ax_energy.plot(ldos_center, energies, color='blue')
    ax_energy.axhline(y=bgmin, color='red', linestyle='--', label='BG Min/Max')
    ax_energy.axhline(y=bgmax, color='red', linestyle='--')
    ax_energy.axvline(x=0, color='black', linestyle='--')
    ax_energy.set_title("LDOS vs Energy", fontsize=16)
    ax_energy.set_xlabel("LDOS (a.u.)", fontsize=14)
    ax_energy.set_ylabel("$E - E_f$ (eV)", fontsize=14)
    ax_energy.tick_params(axis='both', which='major', labelsize=10)

    # Apply sizes to Spatial Axis
    ax_spatial.tick_params(axis='both', which='major', labelsize=10)
    ax_spatial.set_xlabel(ax_spatial.get_xlabel(), fontsize=14)
    ax_spatial.set_ylabel(ax_spatial.get_ylabel(), fontsize=14)
```

Motivation

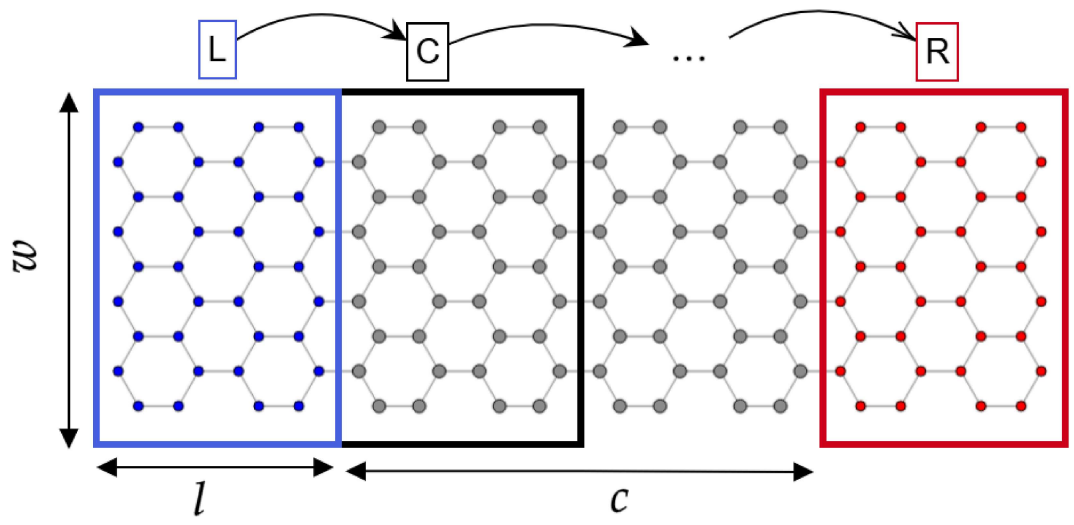
For Quantum Twisting Microscope, the relative angles between layered graphene affect inter-layer transportation, allowing for probing electronic structure of graphene layer or a 'sandwiched' structure.

Focusing on the center of the tip this region would behave a bulk graphene -- no edge effets.
So, can we simulate a graphene leyer of finite width to achieve bulk behavoir in center region?

Creation of reduced structure

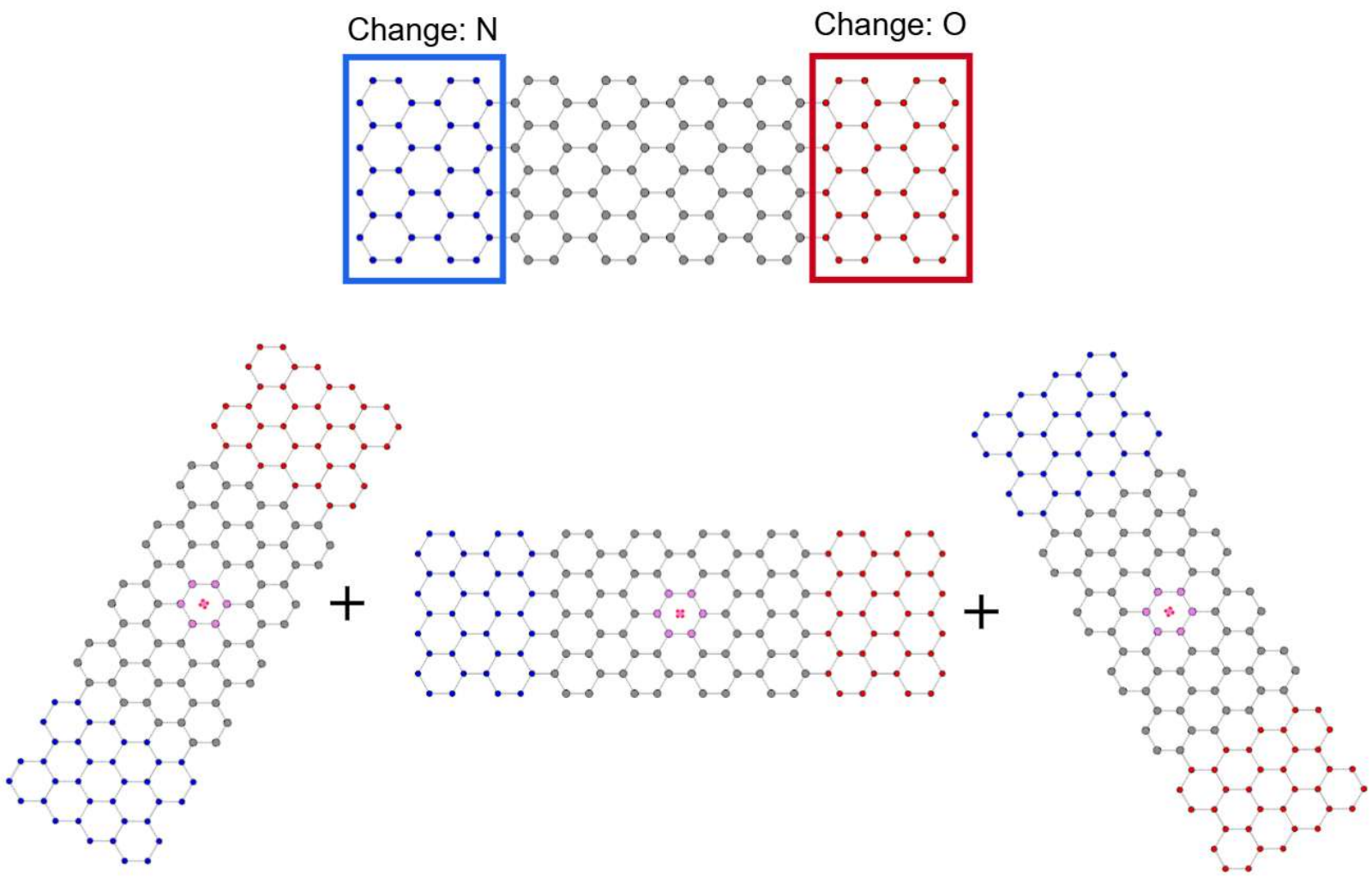
Workflow: Nanoribbon

1. Create electrode: `build_electrode(w, l)`
   - width : number of atoms in *y* (image: 9)
   - length : how many *armchair* in *x* (image: 2)
2. Create center region: `build_nanoribbon(electrode, c)`
   - `electrode` : sisl object
   - center size : how many `electrodes` to use for center region.
   - automatically adds the left/right electrodes



Workflow: Reduced device `build_reduced_device(ribbon, repeat=3)`

3. Change `electrode` atoms
   - left and right has different atoms
4. Copy structure, rotate around center and add to final structure
   - Repeat 3 times for angles : $\{0^\circ,\ \pm 60^\circ\}$



5. Remove atoms overlapping
    - If atoms has same position (tolerance: 0.1 Å)
6. Save atom indecies for electrodes (left: N, right: O)
7. Change all atoms to C

Compatibility issues with $w,\ l,\ c$

Case: Center size too small -- uncomment line 8

In [5]:
```
from utils.structure import build_electrode, build_nanoribbon, build_reduced_device
from utils.plots import mark_electrode
w, l, c = 13, 2, 2
e = build_electrode(w, l)
r = build_nanoribbon(e, c)
d, lr_idx = build_reduced_device(r, e, repeat=3)
atoms_style = []
atoms_style = mark_electrode(lr_idx) # uncomment to mark electrodes
d.plot(axes="xy", atoms_style=atoms_style) # plot device in xy plane
```

```
      Initial number of atoms: 624
Atoms after removing overlaps: 372
```

In [6]:
```
from utils.plots import plot_with_center
plot_with_center(r)
```

Infer dimensions

Ribbon: Geometric center close to high symmetry point (carbon ring center) $W = 5 + 4i$

- If $W < 5$ geometric center is (likely) on a dimer.
- Center region has to satisfy : $2(i+1) = 0 \mod L \rightarrow C = \frac{2(i+1)}{L}$
- Spacing between valid i: $s = \frac{L}{\gcd(2,L)}$
- Lowest valid: $T = -1 \mod s$
- Given i' the next valid is $i = i' + (T - i') \mod s$
  Implemented in `utils.infer_centersize(l,i)`

In [7]:
```
l_idx, r_idx = lr_idx
```

In [8]:
```
from utils import build_electrode, build_nanoribbon, infer_centersize
from utils.plots import plot_with_center, mark_electrode
l, i = 2, 5
l, w, c = infer_centersize(l, i)
# l,w,c = 2, 5, 2 --- IGNORE ---
print(f"Length: {l}, Width: {w}, Center size: {c}, i: {(w-5)//4}")

e = build_electrode(w, l) # width, length
r = build_nanoribbon(e, c) # electrode, center_size
d,lr_idxs = build_reduced_device(r, e, repeat=3) # device, electrode, amount of rotations
plot_with_center(r)
```

```
Length: 2, Width: 25, Center size: 6, i: 5
       Initial number of atoms: 2400
 Atoms after removing overlaps: 1464
```

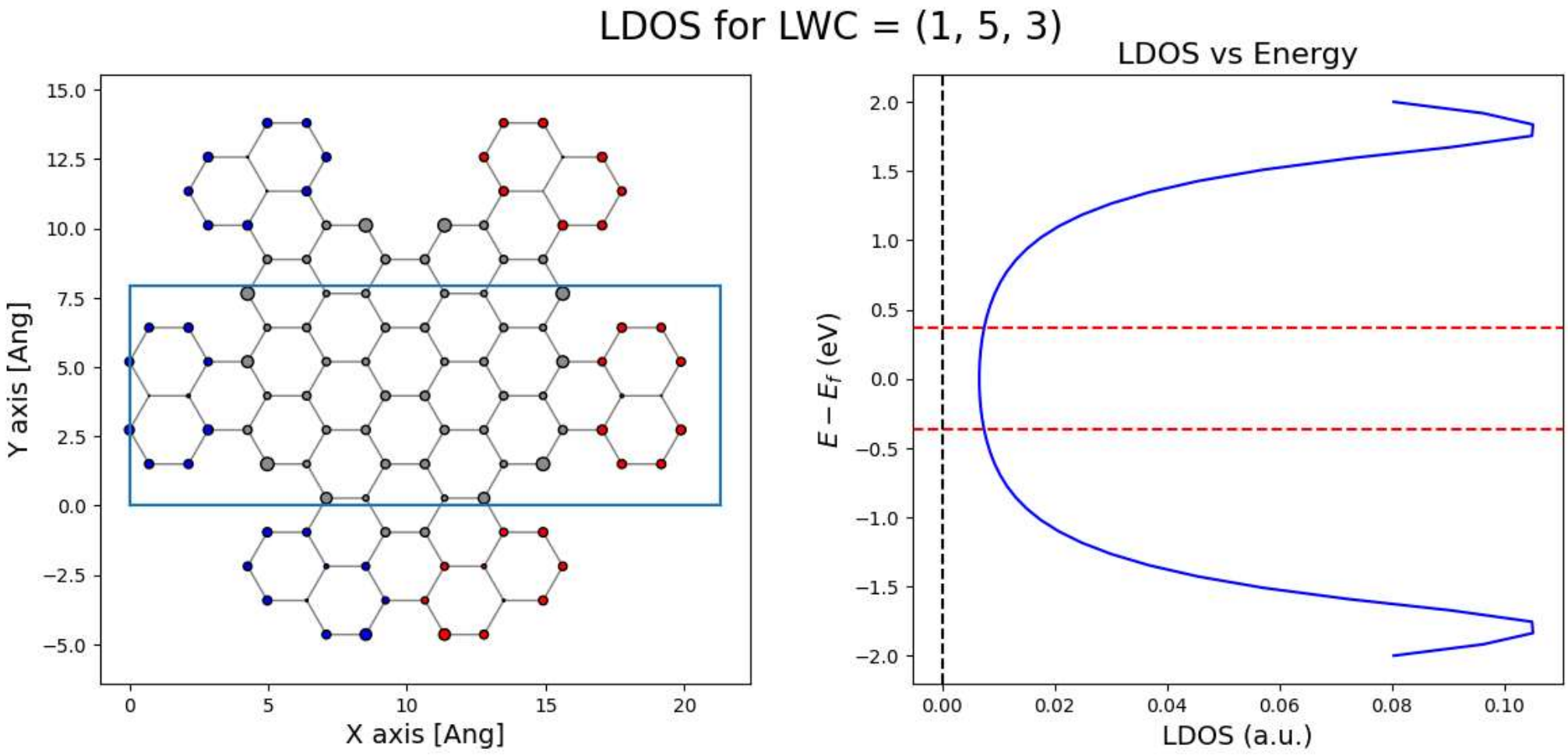
In [9]:
```
a_style = mark_electrode((lr_idxs))
plot_with_center(d, atoms_style=a_style)
```

Why does 'compatible' dimensions matter?

The bad case

Edge states in center region

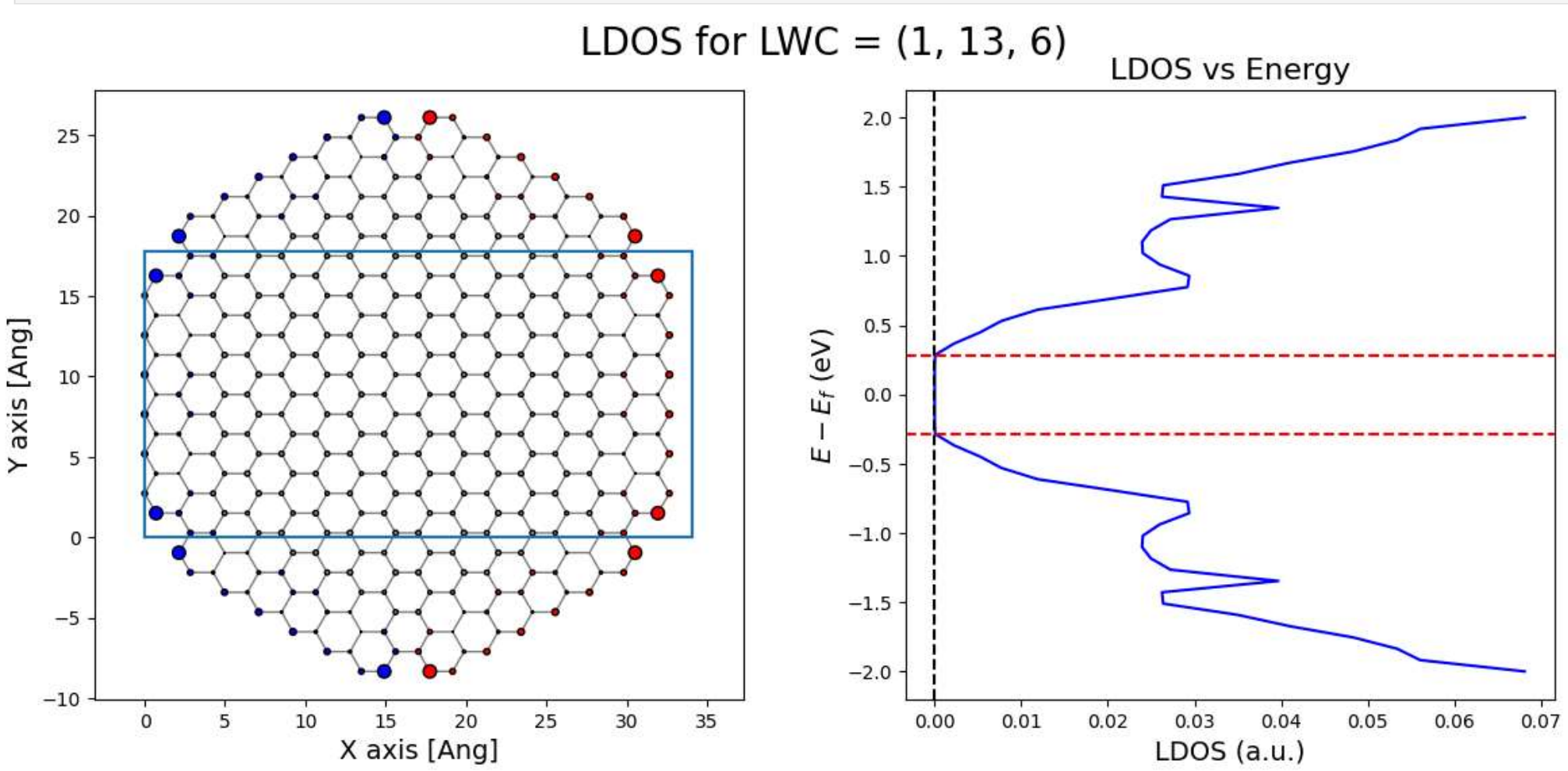
In [10]:
```
lwc = 1, 5, 3
show_ldos_pdos_old(lwc)
```



If we choose *LWC* compatible

Still edge states in electrodes

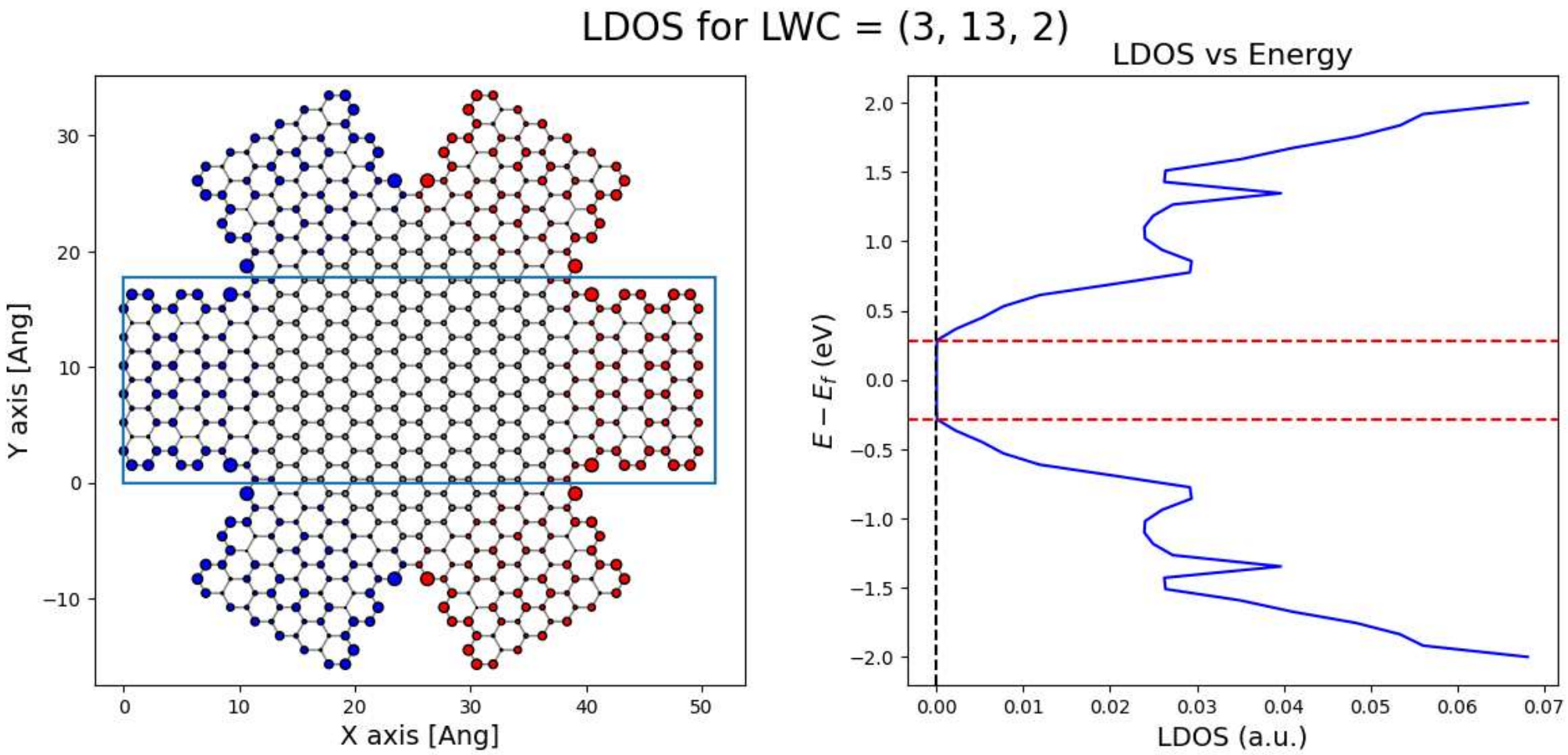
In [11]:
```
lwc = infer_centersize(1, i=6)
# lwc = 1,37,18
lwc = 1,13,6
show_ldos_pdos_old(lwc)
```



Does electrode length remove edge states?

In [12]:
```
lwc = infer_centersize(3, i=2)
print(lwc)
show_ldos_pdos_old(lwc)
```

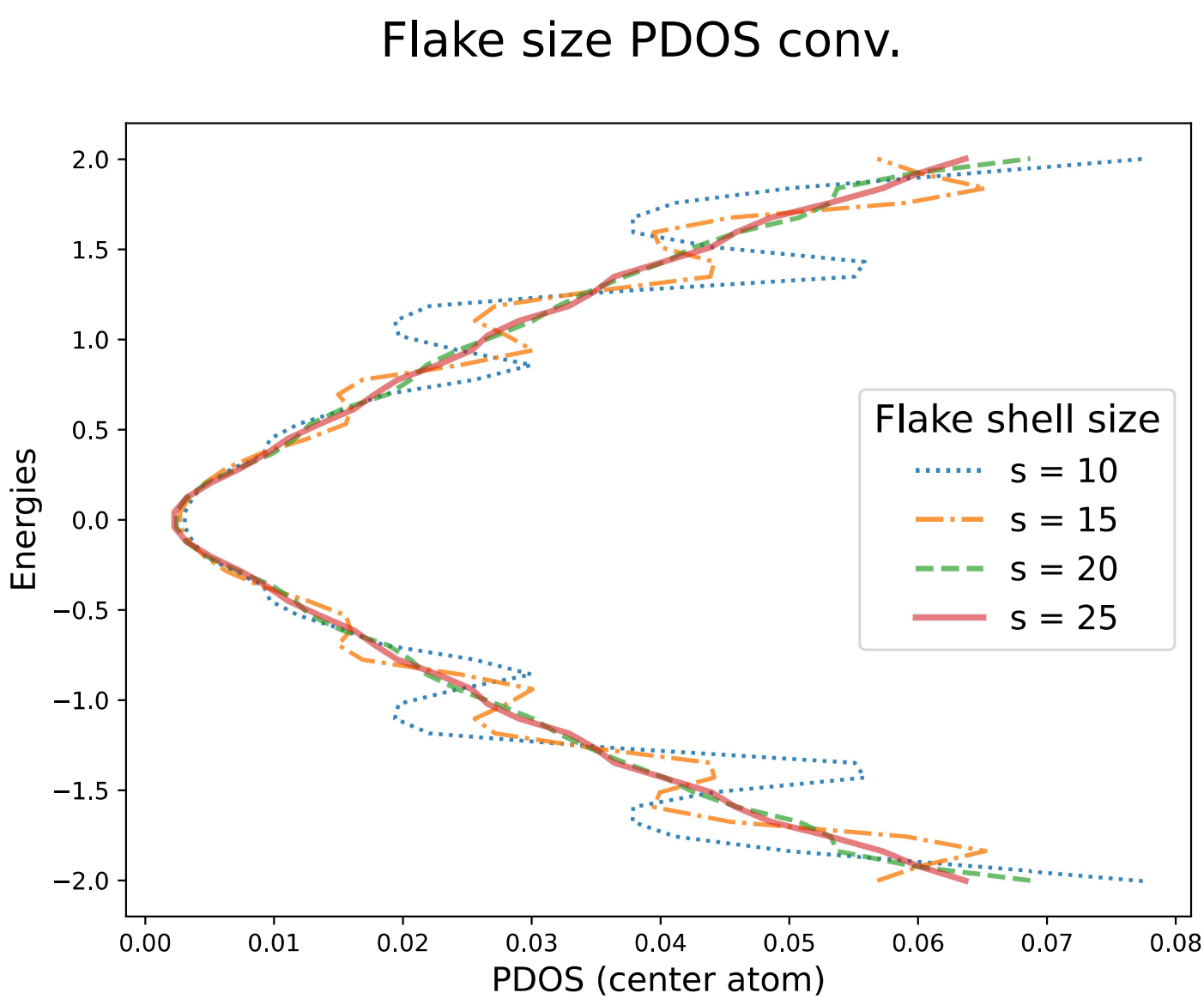
```
(3, 13, 2)
```



Ideally we have the ldos approach the bulk graphene

We can plot the built-in PDOS calculations for a graphene flake of increasing number of shells:

```
# This is a snippet of function
flake = sisl.geom.graphene_flake(size)
H = hamiltonian(flake)
es = H.eigenstate()
pdos = es.PDOS(E=energies).squeeze() # reduce from (1, N, M) -> (N, M)
center_pdos = pdos[0] # for graphene flake the lower indecies are closer to center
plt.plot(center_pdos, energies)
```



What if we modulate the self-energies

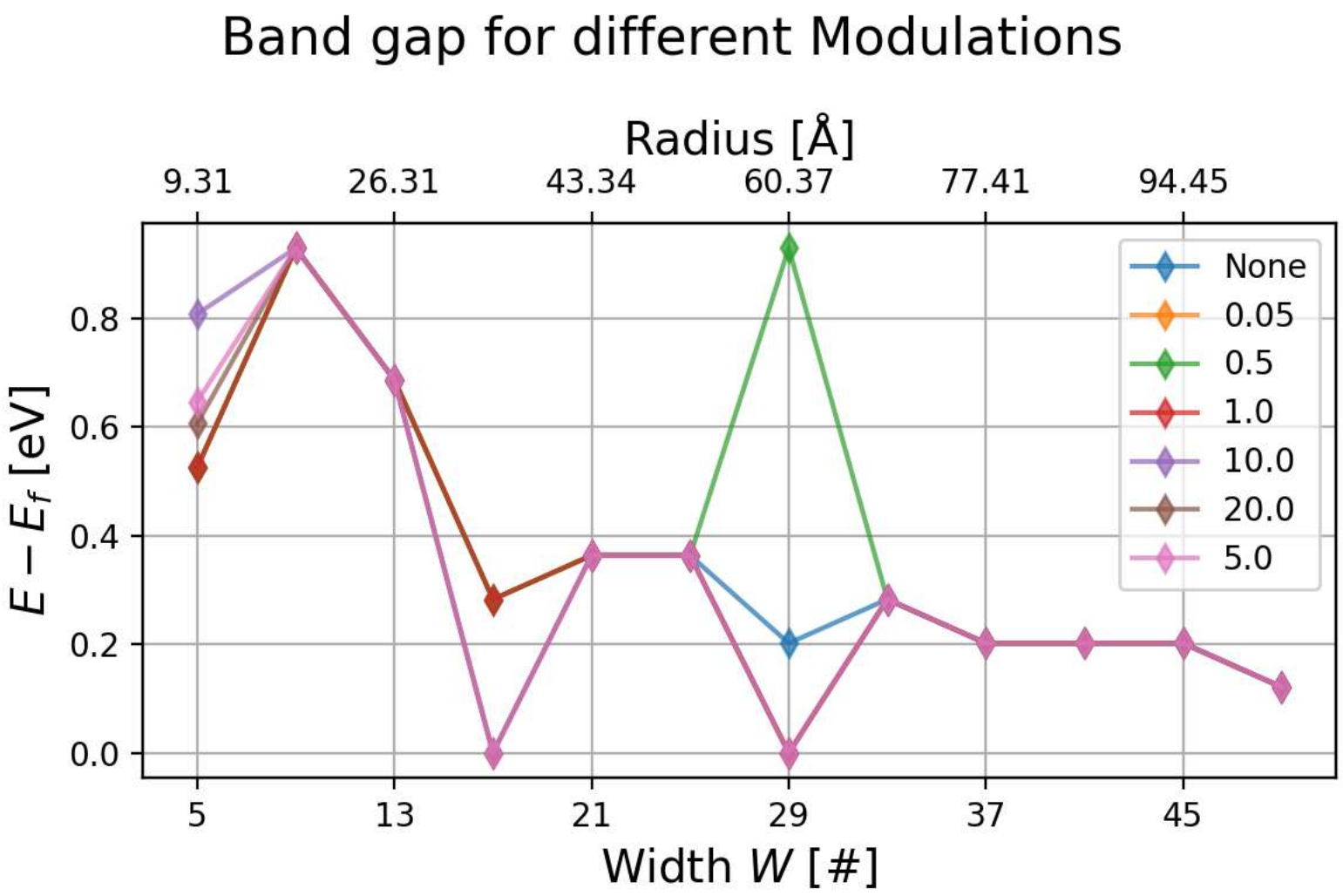
In [13]:
```
from utils.loader import path_finder, parse_lwc, load_datastructure
results_dir = path_finder("QTM") / "results"
lwc_list = [parse_lwc(f) for f in sorted(results_dir.glob("L*_W*_C*"))]
modulations = np.unique([str(m.split("_")[1]) for m in load_datastructure(lwc_list[0])[-1].keys() if m.startswith("bg")]).tolist()
```

```
valid_idxs = [i for i, lwc in enumerate(lwc_list) if load_datastructure(lwc)[-1]["ldos"].shape[-1]>6]
valid_lwcs = [lwc_list[i] for i in valid_idxs]

interact(show_modulation,LWC=valid_lwcs, C=modulations);
```

```
interactive(children=(Dropdown(description='LWC', options=((1, 5, 2), (1, 9, 4), (1, 13, 6), (1, 17, 8), (1, 2…
```

How does changing modulation and size affect the LDOS?



Code deep-dive

The amount of functions and methods make it hard to show everything completely, so here are some highlights for what was difficult and some cool quality of life implementations. Finally some insights into the algorithms

1. Finding band gap

Not perfect, but any other method found turned out worse when I tried it (false BG regions, LWC=(1,29,18), C=0.05) The process is as follows:

```
# mask for energies within energy window around Fermi level (0 eV)
near_fermi = np.abs(energies) <= energy_window

# mask for near_fermi AND within some lower percentile of values of ldos
is_gap = (norm_ldos <= atol) & near_fermi
bgmin = energies[is_gap].min() if np.any(is_gap) else 0.0
bgmax = energies[is_gap].max() if np.any(is_gap) else 0.0
```

---

2. `diagonal_of_inverse()`

If we are ony concerned with diagonal of Greens, then solving for that is sufficient:

```
# for sparse input matrix `M`
from scipy.sparse.linalg import splu
n = M.shape[0]
lu = splu(M)

if sites is not None: m = len(sites)
else: m = n

diag = np.empty(m, dtype=complex)
for i, idx in enumerate(sites): # all the sites of interest; 0 <= i < Na, for all i in sites
    ei = np.zeros(n)
    ei[idx] = 1.0
    xi = lu.solve(ei)
    diag[i] = xi[idx]
return diag
```

---

3. How can we modulate self-energies?

We want some change to the self-energies to reduced the edge states: $\Sigma_{(L/R)} \leftarrow \Sigma_{(L/R)} + i\mathrm{LDOS}(E=0)C$

```
# used in ldos rountine for modulating the self energy
def make_scaling_matrix(ldos0):
    D = np.sqrt(ldos0) # shape (Na,)
    return D[:, None] * D[None, :] # shape (Na, Na) -- diagonal is exactly LDOS(E=0)
#
# ....
#
# INSIDE: the `multi_LDOS()`
#
# Pre-compute LDOS(E=)
if modulate_SE is not None: #
    C = float(modulate_SE) # ensure float
    if LDOS_E0 is None: # compute LDOS at E=0
        LDOS_E0 = np.zeros((Nk, Na), dtype=complex)
        E0 = 0.0 + 1j*eta

        for ik, kvec, in enumerate(kpts):
            Hk = H_D.Hk(k=kvec, format=form, dtype=complex)
            Sk = H_D.Sk(k=kvec, format=form, dtype=complex)

            SE_pair = lr_energies(electrode=SE, En=E0, kvec=kvec)
            Hk_lr = add_lr_energies(Hk.copy(), SE_pair, lr_indices)

            invG = Sk*E0 - Hk_lr
            diagG = diagonal_of_inverse(invG, sites=None) # always compute full diag for E=0
            LDOS_E0[ik, :] = - np.imag(diagG) / np.pi
#
# ...
#
    for ik, kvec in enumerate(kpts):
        Hk = H_D.Hk(k=kvec, format=form, dtype=complex)
        Sk = H_D.Sk(k=kvec, format=form, dtype=complex)
        if modulate_SE is not None:
            scale = make_scaling_matrix(LDOS_E0[ik, :]) * C
        for ie, E in enumerate(tqdm(energies, desc=f"Energy for ik={ik}", **TQDM_KWARGS)):
            En = E + 1j*eta

            SigmaL, SigmaR = lr_energies(electrode=SE, En=En, kvec=kvec)
            if modulate_SE is not None:
                NL, _ = SigmaL.shape
                NR, _ = SigmaR.shape
                SigmaL = SigmaL + 1j*scale[:NL, :NL]
                SigmaR = SigmaR + 1j*scale[-NR:, -NR:]

            Hk_lr = add_lr_energies(Hk.copy(), (SigmaL, SigmaR), lr_indices)
            invG  = Sk*En - Hk_lr
            diagG = diagonal_of_inverse(invG, sites=sites) # compute only needed sites if specified
            all_ldos[ik, ie, :] = -np.imag(diagG) / np.pi

    return all_ldos, LDOS_E0
```

All this can be taken care of when running the LDOS calcualtion:

```
def multi_LDOS(device: sisl.Geometry,
               electrode: sisl.Geometry,
               lr_indices: tuple[np.ndarray, np.ndarray],
               *,
               energies: np.ndarray | list = [0.0],
               Nk: int = 1,
               eta: float = 1e-5,
               form: str = "csc",
               modulate_SE: float | None = None,
               LDOS_E0: np.ndarray | None = None,
               sites: Iterable[int] | None = None) -> np.ndarray:

    # ---- This is only a verbatim ----
    return all_ldos, LDOS_E0
```

Further improvements

- Utilizing parallelization
- Using numba -- `jit()` and `njit()`
- GPU for matrix operations (NVIDIA Cuda)

What is the bottleneck?

Running the script for computing LDOS for 5 different C & W > 33 : ~ 1.5 hours...
Inverse: Taking the inverse of a 2000×2000 $G^{(-1)}$ is not efficient

- Idea: LDOS only concerned with diagonal of Greens function: G_{ii}
- LU factorization and sparse matricies
- Further: '*batching*' the calculations to reduce computation time (not yet realized)

But what does the computer tell us is the slowest process?
Turns out it is the `sisl.physics.self_energy` module (specifically the `self_energy_lr()` method)

In [14]:
```
%load_ext line_profiler
from utils import load_datastructure, multi_LDOS, lr_energies
```

In [15]:
```
i = -1
electrode, ribbon, device, data = load_datastructure(lwc_list[i]); lwc_list[i]
device.na
```

Out[15]:
```
2328
```

In [16]:
```
lprun_args = dict(device=device,
                  electrode=electrode,
                  energies=np.linspace(-2, 2, num=1),
                  lr_indices=data["lr_idx"],
                  Nk=1,
                  form="csc",
                  eta=1e-5,
                  modulate_SE=None,
                  LDOS_E0=None,
                #   sites=None
                  sites=find_nearest_atoms(device.xyz, device.center(), neighbours=6)
)
%lprun -f multi_LDOS -f lr_energies multi_LDOS(**lprun_args)
```

```
warn:0: SislWarning:

Geometry.close_sc has been passed an 'atoms' argument together with an R value larger than the orbital ranges. If used together with 'sparse
-matrix.construct' this can result in wrong couplings.

kvecs:   0%|          | 0/1 [00:00<?, ?it/s]
Energy for ik=0:   0%|          | 0/1 [00:00<?, ?it/s]
Sparse diagonal solves:   0%|          | 0/6 [00:00<?, ?it/s]
remember that output is now a tuple `all_ldos, LDOS_E0`
```

```
Timer unit: 1e-09 s

Total time: 3.50606 s
File: /home/bastian/projects/QuantumTwistingMicroscope/QTM/utils/energies.py
Function: lr_energies at line 96

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    96                                           def lr_energies(electrode: sisl.Geometry | sisl.Hamiltonian | RecursiveSI,
    97                                                           En: complex = 1e-5j, kvec: Sequence = [0,0,0]) -> Tuple[np.ndarray, np.ndar
ray]:
    98                                               """Compute the left/right self-energy.
    99
   100                                               Parameters
   101                                               ----------
   102                                               electrode : Geometry  |  Hamiltonian  |  RecursiveSI
   103                                                   The electrode to find self-energies.
   104                                               En : complex, optional
   105                                                   The energy for the self-energy calculation, by default 1e-5j
   106                                               kvec : list, optional
   107                                                   The kpoint for the self-energy, by default [0,0,0]
   108
   109                                               Returns
   110                                               -------
   111                                               tuple[np.ndarray, np.ndarray]
   112                                                   Left and right self-energies
   113
   114                                               Raises
   115                                               ------
   116                                               TypeError
   117                                                   If electrode is of invalid type
   118                                               """
   119         1       1741.0   1741.0      0.0      if not isinstance(kvec, list):
   120         1       9900.0   9900.0      0.0          kvec = list(kvec)
   121         1       3483.0   3483.0      0.0      if isinstance(electrode, sisl.Geometry):
   122                                                   H_0 = hamiltonian(electrode)
   123                                                   SE = RecursiveSI(H_0, infinite="+A")
   124         1       2383.0   2383.0      0.0      elif isinstance(electrode, sisl.physics.RecursiveSI):
   125         1        733.0    733.0      0.0          SE = electrode
   126                                               elif isinstance(electrode, sisl.Hamiltonian):
   127                                                   SE = RecursiveSI(electrode, infinite="+A")
   128                                               else:
   129                                                   raise TypeError("argument 'electrode' must be Geometry, Hamiltonian, or RecursiveS
I")
   130         1 3506040182.0 3.51e+09    100.0      SE_L, SE_R = SE.self_energy_lr(E=En, k=kvec)
   131         1        550.0    550.0      0.0      return SE_L, SE_R

Total time: 5.09216 s
File: /home/bastian/projects/QuantumTwistingMicroscope/QTM/utils/energies.py
Function: multi_LDOS at line 257

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
   257                                           def multi_LDOS(device: 'sisl.Geometry',
   258                                                          electrode: 'sisl.Geometry',
   259                                                          lr_indices: tuple[np.ndarray, np.ndarray],
   260                                                          *,
   261                                                          energies: np.ndarray | list = [0.0],
   262                                                          Nk: int = 1,
   263                                                          eta: float = 1e-5,
   264                                                          form: str = "csc",
   265                                                          modulate_SE: float | None = None,
   266                                                          LDOS_E0: np.ndarray | None = None,
   267                                                          sites: Iterable[int] | None = None) -> np.ndarray:
   268
   269
   270                                               """Compute LDOS.
   271                                               if modulate_SE is a float, self-energies are modulated with
   272                                               `Sigma_{L/R} + i*LDOS(E=0)*modulate_SE.
   273
   274                                               Paramters
   275                                               ---
   276                                               modulate_SE : float or None
   277                                                   - None -> No modulation (original behavior)
   278                                                   - float -> modulation strength C
   279
   280
   281                                               Returns
   282                                               -------
   283                                               LDOS : ndarray of shape (Nk, NE, Na)
   284                                                   The Ldos for the number of k points, energies and atoms/sites: Nk, NE, Na, respecti
vely.
   285                                               """
   286         1      15034.0  15034.0      0.0      Na = len(device) # number of atoms
   287         1       2566.0   2566.0      0.0      energies = np.asarray(energies) # ensure energies is a ndarray
   288         1        550.0    550.0      0.0      Ne = len(energies) # number of energies
   289         1        550.0    550.0      0.0      k_direction = [1, Nk, 1] # direction to sample k
   290
   291                                               # Determine sites to compute LDOS for
   292         1        733.0    733.0      0.0      if sites is not None:
   293         1      81859.0  81859.0      0.0          if (not np.all(0 <= (sites) & (sites < Na))): # compute for subset of sites
```

```
   294                                                 raise ValueError("'sites' must be None or list of integers within [0, Na).")
   295
   296
   297         1 1293885757.0 1.29e+09     25.4      H_D = hamiltonian(device)
   298         1     727284.0 727284.0      0.0      H_D.set_nsc([1,1,1])
   299
   300
   301         1     514159.0 514159.0      0.0      kpts = sisl.MonkhorstPack(H_D, k_direction).k
   302
   303         1  102982941.0 1.03e+08      2.0      H_0 = hamiltonian(electrode)
   304         1    7794602.0 7.79e+06      0.2      SE = RecursiveSI(H_0, infinite="+A")
   305
   306                                              # Precompute LDOS(E=0) if needed
   307         1        550.0    550.0      0.0      if modulate_SE is not None: #
   308                                                  C = float(modulate_SE) # ensure float
   309                                                  if LDOS_E0 is None: # compute LDOS at E=0
   310                                                      LDOS_E0 = np.zeros((Nk, Na), dtype=complex)
   311                                                      E0 = 0.0 + 1j*eta
   312
   313                                                      for ik, kvec, in enumerate(tqdm(kpts, desc="LDOS E=0", **TQDM_KWARGS)):
   314                                                          Hk = H_D.Hk(k=kvec, format=form, dtype=complex)
   315                                                          Sk = H_D.Sk(k=kvec, format=form, dtype=complex)
   316
   317                                                          SE_pair = lr_energies(electrode=SE, En=E0, kvec=kvec)
   318                                                          Hk_lr = add_lr_energies(Hk.copy(), SE_pair, lr_indices)
   319
   320                                                          invG = Sk*E0 - Hk_lr
   321                                                          diagG = diagonal_of_inverse(invG, sites=None) # always compute full diag fo
r E=0
   322                                                          LDOS_E0[ik, :] = - np.imag(diagG) / np.pi
   323         1        367.0    367.0      0.0      if (sites is not None): # override number of sites if only considering subset
   324         1        917.0    917.0      0.0          Na = len(sites)
   325                                              # Compute all LDSO (optionally modulated)
   326         1       2292.0   2292.0      0.0      all_ldos = np.zeros(shape=(Nk, Ne, Na), dtype=float)
   327         2   22491064.0 1.12e+07      0.4      for ik, kvec in enumerate(tqdm(kpts, desc="kvecs", **TQDM_KWARGS)):
   328         1   40264318.0 4.03e+07      0.8          Hk = H_D.Hk(k=kvec, format=form, dtype=complex)
   329         1    2858351.0 2.86e+06      0.1          Sk = H_D.Sk(k=kvec, format=form, dtype=complex)
   330
   331         1        550.0    550.0      0.0          if modulate_SE is not None:
   332                                                      scale = make_scaling_matrix(LDOS_E0[ik, :]) * C
   333
   334         2   23754047.0 1.19e+07      0.5          for ie, E in enumerate(tqdm(energies, desc=f"Energy for ik={ik}", **TQDM_KWARGS)):
   335         1      35292.0  35292.0      0.0              En = E + 1j*eta
   336
   337         1 3506078500.0 3.51e+09     68.9              SigmaL, SigmaR = lr_energies(electrode=SE, En=En, kvec=kvec)
   338         1        733.0    733.0      0.0              if modulate_SE is not None:
   339                                                          NL, _ = SigmaL.shape
   340                                                          NR, _ = SigmaR.shape
   341                                                          SigmaL = SigmaL + 1j*scale[:NL, :NL]
   342                                                          SigmaR = SigmaR + 1j*scale[-NR:, -NR:]
   343
   344         1   58342089.0 5.83e+07      1.1              Hk_lr = add_lr_energies(Hk.copy(), (SigmaL, SigmaR), lr_indices)
   345         1     810333.0 810333.0      0.0              invG  = Sk*En - Hk_lr
   346         1   31326816.0 3.13e+07      0.6              diagG = diagonal_of_inverse(invG, sites=sites) # compute only needed sites if s
pecified
   347         1      43633.0  43633.0      0.0              all_ldos[ik, ie, :] = -np.imag(diagG) / np.pi
   348
   349         1      25300.0  25300.0      0.0      if in_notebook():
   350         1     115042.0 115042.0      0.0          print("remember that output is now a tuple `all_ldos, LDOS_E0`")
   351                                              # if LDOS_E0 is None:
   352                                              #     return all_ldos, None
   353                                              # else:
   354         1        550.0    550.0      0.0      return all_ldos, LDOS_E0
```

Future development:

1. Speed up `sisl.physics.self_energy`
2. Parallelization and Cuda
3. Numba
4. Modulation implementaiton has little effect